

CSCI 232

232 Algorithms & Data Structures

LECTURE: 15

J. L. Pach

TODAY'S AGENDA

- Space Complexity $S(n)$
  - Input, Auxiliary and Total Space
  - Memory Model
  - Sum over Time vs Maximum over Time
- Space of Recursive Algorithms
  - The Call Stack
  - Space Recurrences
- Asymptotic Notation for $S(n)$
- Time–Space Trade-off

SPACE

Complexity

FROM T(N) TO S(N)

- So far we asked: **how much work** does an algorithm do? → $T(n)$
- Now we ask: **how much memory** does it need to do this work? → $S(n)$
- Both are functions of the input size n, both are described with the **same** asymptotic notation (Θ, O, Ω).
- What changes is **what we count** and **how we combine the counts**.

	$T(n)$	$S(n)$
We count	primitive operations	memory cells (words)
Over time we take	the **sum**	the **maximum**

Input, Auxiliary and Total Space

- **Input space** — memory occupied by the input itself. For an array of n elements it is $\Theta(n)$, regardless of the algorithm.
- **Auxiliary space** — *extra* memory the algorithm uses while running: local variables, additional arrays, the recursion stack.
- **Total space** — input space + auxiliary space.

$$S_{total}(n) = \underbrace{S_{input}(n)}_{\text{given}} + \underbrace{S_{aux}(n)}_{\text{algorithm's choice}}$$

> **Convention in this course:** when we write $S(n)$ we mean the **auxiliary space** $S_{aux}(n)$.
> Reason: the input is the same for every algorithm solving the problem, so only the auxiliary space tells algorithms apart.

AUXILIARY SPACE — WHAT COUNTS

Counted in $S(n)$:

- local variables and loop counters
- additional arrays / lists / trees created by the algorithm
- the array/structure that the algorithm **builds and returns**
- stack frames of recursive calls

Not counted in $S(n)$:

- the input array A passed to the algorithm
- the program code itself

> Some textbooks do not count the output either. The result is the same up to the convention, so **always state which one you use**.

MEMORY MODEL

As in the time analysis, where we replaced the cost of each primitive operation by a constant, we simplify memory:

- one variable (`int`, `char`, `bool`, pointer, index) occupies **one cell** → $\Theta(1)$
- an array of n elements occupies n cells → $\Theta(n)$
- a matrix $n \times n$ occupies n^2 cells → $\Theta(n^2)$
- one stack frame of a function call occupies a **constant** number of cells → $\Theta(1)$

> **Why can we ignore the exact sizes?** If every cell occupies between b_{min} and b_{max} bytes (positive constants), then
> $b_{min} \cdot \text{cells} \leq \text{bytes} \leq b_{max} \cdot \text{cells}$, so the order of growth does not change.
> This is the same argument we used for setting $c_1 = \cdots = c_7 = 1$ in $T(n)$.

MEMORY MODEL — LIMITS OF THE MODEL

- We assume that a number fits into **one cell**, no matter its value. Storing the value $n = 10^6$ does **not** cost 10^6 cells.
- Count the **number** of variables, not their **values**.
- The assumption breaks for numbers that grow with n (e.g. $n!$ or huge integers). We do not consider such cases in this course.
- Exact bytes, alignment, compiler optimizations and garbage collection are **out of scope**.

SUM OVER TIME VS MAXIMUM OVER TIME

Time is **consumed**: every executed operation adds to the total.

Memory is **reused**: a cell that is released can be used again.

$$T(n) = \sum_{i=1}^{7} c_i \cdot \mathrm{count}_i(n) \qquad\qquad S(n) = \max_t \ \mathrm{mem}(t)$$

where $\mathrm{mem}(t)$ is the number of cells occupied at the moment t of the execution.

> A loop with n iterations costs $\Theta(n)$ **time**, but if every iteration reuses the same variables, it costs only $\Theta(1)$ **space**.

Example 1 — Constant Space

m_1 = int ; m_2 = float ; m_3 = double ; m_x = type ;

```c
int sumArray(const int A[], int n)
{
    int sum = 0;                    // 1 cell
    for (int i = 0; i < n; i++)     // 1 cell
        sum += A[i];
    return sum;
}
```



A is the input → not counted in $S_{aux}(n)$

n is a parameter → not counted

sum → 1 cell

i (loop counter) → 1 cell

$$S_{aux}(n) = 1 + 1 = 2$$

Asymptotic simplification:

$$S_{aux}(n) = 2m = 2 \quad \text{where } m = 1$$

$$S(n) = \Theta(1)$$

> The loop runs n times, but it creates no new cells.

$$T(n) = \Theta(n) \qquad S(n) = \Theta(1)$$

EXAMPLE 2 — LINEAR SPACE

```c
int* copyArray(const int A[], int n) {
    int* B = new int[n];              // n cells
    for (int i = 0; i < n; i++)       // 1 cell
        B[i] = A[i];
    return B;
}
```



- $S(n) = n + \text{const}$, because `B` has n cells and the rest is a constant number of variables.
- Same simplification as for $T(n)$: constants and lower-order terms disappear.

$$T(n) = \Theta(n) \qquad S(n) = \Theta(n)$$

> Under our convention the returned array **is counted**.

Example 3 — Allocation Inside a Loop

```
for (int i = 0; i < n; i++)
{
    int* tmp = new int[n];
    // ... use tmp ...
    delete[] tmp;                  // released in every iteration
}
```



- At any moment only **one** array exists, so $S(n) = \Theta(n)$ (a maximum, not a sum).
- Without `delete[]` (a memory leak) the arrays pile up, so $S(n) = \Theta(n^2)$.

> $S(n)$ depends on **when memory is released**, not only on how much is allocated in total.

SPACE OF

Recursive Algorithms

THE CALL STACK

Every function call creates a **stack frame** holding parameters, local variables and the return address. The frame exists until the call returns.

```c
int fact(int n)
{
  if (n <= 1)
    return 1;
  return n * fact(n - 1);
}
```



```
fact(1)   <- top
fact(2)
fact(3)
fact(4)
fact(5)   <- bottom
```

`fact(5)` has 5 frames on the stack at the deepest point. Each frame is a constant number of cells.

$$S(n) = \text{depth of recursion} \times \text{size of one frame} = n \cdot \Theta(1) = \Theta(n)$$

NESTED VS SEQUENTIAL CALLS

- **Nested** calls (a call inside a call) are alive at the same time, so their frames **add up**.
- **Sequential** calls (one returns before the next starts) reuse the same stack space, so we take the **maximum**.

```c
void f(int n)
{           // S(n) = ?
   if (n <= 1)
     return;
   f(n / 2);            // returns, frame released
   f(n / 2);            // reuses the same stack space
}
```



Both calls have the same depth. Only the **longest chain of nested calls** counts, which is $\log n$ frames, so $S(n) = \Theta(\log n)$. The time is $T(n) = \Theta(n)$.

SPACE RECURRENCES

Just as $T(n)$ gives a recurrence, so does $S(n)$. The shape is the same, but sequential calls are combined with **max**, not `+`.

Algorithm	Recurrence	Solution
Recursive factorial	$S(n) = S(n-1) + \Theta(1)$	$\Theta(n)$
Recursive binary search	$S(n) = S(n/2) + \Theta(1)$	$\Theta(\log n)$
Merge sort	$S(n) = S(n/2) + \Theta(n)$	$\Theta(n)$
Naive Fibonacci `fib(n-1)+fib(n-2)`	$S(n) = S(n-1) + \Theta(1)$	$\Theta(n)$

ASYMPTOTIC

Notation for S(n)

THE SAME NOTATION, A DIFFERENT FUNCTION

- Θ, O, Ω are defined for **functions**. They do not care whether the function counts operations or memory cells.
- So the definitions from the previous part stay unchanged. We only substitute $S(n)$ for $f(n)$:

$$S(n) = O(g(n)) \iff \exists\, c, n_0 > 0 :\ 0 \leq S(n) \leq c\, g(n)\ \ \forall\, n \geq n_0$$

- Example: $S(n) = 4n + 12$ gives $S(n) = \Theta(n)$, and therefore also $O(n)$ and $\Omega(n)$.
- It is also true that $S(n) = O(n^2)$, but this bound is **not tight**.

CASE AND NOTATION ARE INDEPENDENT

Step 1: which function?

- best-case $S(n)$
- average-case $S(n)$
- worst-case $S(n)$

Step 2: which bound?

- Θ: tight
- O: upper
- Ω: lower

> O does **not** mean "worst case" and Ω does **not** mean "best case". First choose the case, then describe its growth.

EXAMPLE — QUICKSORT: S(N) DEPENDS ON THE CASE

```c
void quicksort(int A[], int p, int r) {
    if (p < r) {
        int q = partition(A, p, r);   // in place, Θ(1) extra cells
        quicksort(A, p, q - 1);
        quicksort(A, q + 1, r);
    }
}
```



The extra memory is the **recursion stack**, so $S(n)$ = depth of recursion.

- Balanced partitions: depth $\log n$, so $S_{best}(n) = \Theta(\log n)$.
- Worst partitions (sorted input, bad pivot): depth n, so $S_{worst}(n) = \Theta(n)$.
- Recursing into the smaller part first (and looping over the larger one) guarantees depth $\leq \log n$, so $S_{worst}(n) = \Theta(\log n)$. (CLRS Problem 7-4)

IN-PLACE ALGORITHMS

An algorithm is **in place** if it uses only $S(n) = \Theta(1)$ extra cells (a looser definition allows $O(\log n)$, which includes quicksort).

Algorithm	$T_{worst}(n)$	$S(n)$	In place?
Insertion sort	$\Theta(n^2)$	$\Theta(1)$	yes
Heapsort	$\Theta(n \log n)$	$\Theta(1)$	yes
Quicksort	$\Theta(n^2)$	$\Theta(\log n)$ to $\Theta(n)$	by the looser definition
Merge sort	$\Theta(n \log n)$	$\Theta(n)$	no

TIME–SPACE

Trade-off

SAME PROBLEM, DIFFERENT (T, S)

Often we can spend more memory to save time, or the other way round.

Problem	Approach	$T(n)$	$S(n)$
Duplicates in an array	two nested loops	$\Theta(n^2)$	$\Theta(1)$
	sort first (heapsort), scan	$\Theta(n \log n)$	$\Theta(1)$
	hash set	$\Theta(n)$ expected	$\Theta(n)$
Fibonacci F_n	naive recursion	$\Theta(\varphi^n)$	$\Theta(n)$
	memoization	$\Theta(n)$	$\Theta(n)$
	iteration with 2 variables	$\Theta(n)$	$\Theta(1)$

> There is no "best" algorithm without stating which resource we optimize.

EXERCISES

Find $(T(n), S(n))$ for each fragment (the input is not counted):

```
// (a)
int m = A[0];
for (int i = 1; i < n; i++)
  if (A[i] > m)
    m = A[i];
```

```
// (b)
int* B = new int[n];
for (int i = 0; i < n; i++)
  B[n - 1 - i] = A[i];
```

```
// (c)
int g(int n) {
  return n <= 0 ? 0 : 1 + g(n - 1);
}
```

```
// (d)
int* M = new int[n * n];
for (int i = 0; i < n * n; i++)
  M[i] = 0;
```

SUMMARY

- $S(n)$ is the **maximum** memory in use at any moment, while $T(n)$ is the **sum** of the work done.
- In this course $S(n)$ means **auxiliary space**: the input is not counted, the output is.
- Variables give $\Theta(1)$, arrays give $\Theta(n)$, recursion gives $\Theta(\text{depth})$.
- The same asymptotic notation is used for $T(n)$ and $S(n)$, and the case (best, average, worst) is chosen independently of the symbol.
- Many problems have a trade-off between $T(n)$ and $S(n)$.

THANK

You